

Classroom Video Attendance Roadmap

Classroom video attendance system

An engineering roadmap for automated face recognition attendance

0. How to read this document

This is a build plan for a system where a teacher records a short video of a classroom, uploads it, and the system automatically produces an attendance record.

The document is organised into eleven phases. Each phase contains four things:

- What it does** — the functionality in plain language, and why it exists.
- Integration contract** — exactly what the phase consumes and what it produces, so phases connect cleanly.
- Definition of done** — how you know you can move on.
- The prompt** — a complete, copy-paste brief for a coding agent such as Codex or Antigravity.

Two prompts sit outside the phases and must be supplied with **every** phase prompt:

- The **global prompt** (Section 4) — the project contract. Stack, conventions, file layout, rules. This is what keeps ten separately-generated phases from turning into ten incompatible codebases.
- The **UI prompt** (Section 5) — the design contract. Only needed for phases that touch the interface (0, 1, 8, 9).

How to feed it to the agent: paste the global prompt, then the UI prompt if relevant, then the phase prompt. Do one phase per session. Do not start a phase until the previous phase’s definition of done is met.

A note on expectations

Build this to learn, and treat the accuracy number as the finding, not the failure. The honest expectation for a first working version on real classroom video is somewhere between 85% and 96% of students correctly identified per session. Phase 7 exists to measure that number precisely, and Phase 8 exists to make the system useful despite it.

If you reach 99%+ on genuine back-row students in a 90-person hall, you have built something the commercial market has not, and you should write that up.

1. The capture protocol

Everything downstream is capped by what the camera collects. A better model cannot recover pixels that were never recorded. This section is therefore the highest-leverage part of the entire project — read it twice.

1.1 The core problem

A face needs roughly 112 x 112 pixels of aligned crop to embed reliably. Work backwards from that:

Setting	Back-row face width at 10 m	Usable?
1080p (1920 x 1080)	approximately 20-28 px	No
1440p (2560 x 1440)	approximately 30-38 px	Marginal
4K (3840 x 2160)	approximately 45-58 px	Yes, with upscaling

Record in 4K. This single setting matters more than every model choice in this document combined.

1.2 The protocol

Camera settings

- Resolution: 3840 x 2160 (4K). If the phone cannot, use the highest available and accept that back rows will fail.
- Frame rate: 30 fps. Do not use 60 fps — it halves exposure time per frame and adds noise in typical classroom lighting.

- Stabilisation: on.
- Before recording, tap to focus on the middle rows, then lock focus and exposure (AE/AF lock). Without the lock, the camera hunts during the pan and produces a run of blurred frames.

Position

- Stand at the front of the room, facing the students.
- Get elevated if you can — a podium step, or simply hold the phone above head height and tilt down.

Elevation is the single largest occlusion fix available. At eye level, back-row heads are hidden behind front-row heads. Raising the camera by 40-50 cm can recover a large fraction of the back rows at zero cost.

Motion

- One slow, continuous horizontal pan: left to right.
- Pause 3-5 seconds.
- Second pan: right to left.
- Total duration for a 90-student hall: 50-70 seconds.
- Pan speed: a 90-degree sweep should take at least 12-15 seconds. Faster than this produces motion blur, and blurred faces are worse than absent faces because they generate confident wrong matches.

The second pan is not redundant. Students who were looking down during the first pass typically look up when they hear the phone — the second pass catches them. In testing this is worth more than any threshold tuning.

Lighting

- Never pan toward windows. A backlit face is a silhouette and face recognition fails on it completely, not partially.
- If windows are on one wall, position yourself so the windows are behind the camera.

Cooperation

- Announce it: “Attendance video, please look up for a moment.”
- Two seconds of student cooperation is worth more than months of model work.
- This announcement is also your consent moment. Students must know they are being recorded and why.

1.3 File size and upload

A 60-second 4K H.264 clip is roughly 250-400 MB. On college wifi this is a genuine obstacle.

- Record in H.265/HEVC if the phone supports it — roughly half the size at the same quality.
- Do not downscale before upload. That destroys the exact pixels the system depends on.
- Use chunked, resumable, background upload. The teacher submits and walks away; the upload finishes whenever the network allows.

1.4 Pre-flight quality check

The system must judge whether a video is usable **before** spending twenty minutes processing it. A teacher who discovers at 11:40 that the 11:05 video was unusable cannot go back and re-shoot. A teacher told at 11:06 that the pan was too fast can simply record again.

This check runs on 15-20 sampled frames and takes under 30 seconds. It is the highest-value fifty lines of code in the project.

Check	How	Fail message to the teacher
Resolution	ffprobe dimensions	"Record in 4K. Change it in your camera settings and try again."
Duration	ffprobe	"Too short — record at least 30 seconds."
Overall sharpness	median Laplacian variance across sampled frames	"The video is blurry. Hold the phone steadier and pan more slowly."
Backlighting	fraction of frames where the upper third is far brighter than detected face regions	"The windows are behind the students. Move so the windows are behind you."
Camera actually panned	horizontal optical flow, or the drift of the mean detection x-position across frames	"The camera did not move across the room. Pan slowly from one side to the other."
Pan too fast	mean per-frame horizontal displacement above a threshold	"You panned too quickly. Take at least 15 seconds for each sweep."
Face yield	detections in sampled frames versus the expected class size	"Very few faces were found. Check that you are facing the students and the room is lit."
Coverage	detection x-positions should span most of the frame width across the video	"Part of the room may have been missed. Pan the full width."

Fail the upload with the plain-language message and let the teacher re-shoot immediately. Warn but proceed on marginal results.

2. System architecture

2.1 The pipeline

```

Teacher records video (4K, 2 passes, ~60s)
|
v
[1] Chunked upload to object storage
|
v
[2] Job queued; worker picks it up
|
v
[3] Frame extraction (ffmpeg, sample at 4 fps -> ~240 frames)
|
v
[4] Face detection per frame (SCRFD) -> bounding boxes + 5 landmarks
|
v
[5] Quality gate (size, blur, pose, detection score)
|   rejects roughly 70-80% of crops
|
v
[6] Alignment (5-point similarity transform -> 112x112)
|
v
[7] Embedding (ArcFace) -> 512-dim vector per surviving crop
|
v
[8] Clustering (DBSCAN on cosine distance)
|   ~4000 crops collapse into ~90-120 identity clusters
|
v
[9] Per-cluster representative (quality-weighted mean embedding)
|
v
[10] Gallery match against enrolled students (cosine similarity)
|
v
[11] Three-band thresholding
|   confident match -> auto-marked present
|   uncertain -> review queue
|   no match -> unknown person cluster
|
v
[12] Teacher review screen (~15 seconds of confirmation)
|
v
[13] Immutable attendance record committed

```

2.2 Why cluster before matching

The obvious design is: for every detected face in every frame, match it against the gallery and vote. Do not do this. Cluster first, then match once per cluster.

The reason is that a single low-quality crop matched against 90 candidates is a noisy, error-prone decision, and taking 240 noisy votes does not reliably produce a clean answer — correlated errors survive voting. Whereas averaging 40 embeddings of the *same* person before matching produces a representative vector that is dramatically cleaner than any individual crop, because the noise is uncorrelated and averages out while the identity signal reinforces.

Concretely: clustering-then-matching typically converts a per-crop accuracy of around 75% into a per-person accuracy above 93%, using the identical model. This is the highest-value architectural decision in the project.

It also gives you something matching alone cannot: a cluster that matches *nobody* in the gallery is a meaningful signal — an unenrolled person, a visitor, or a student whose enrollment photos are bad.

2.3 Order of operations matters for cost

Quality-gate **before** embedding, not after. A 240-frame video of 90 students produces roughly 20,000 raw detections. Embedding all of them on CPU takes 7-10 minutes. Filtering first cuts it to roughly 4,000 crops and 90 seconds. Same result, one-fifth the compute.

2.4 Expected processing time

Environment	Approximate time for a 60s 4K video
Modest CPU VPS (2 vCPU)	12-25 minutes
CPU with ONNX Runtime tuned	6-12 minutes
Serverless GPU (Modal, RunPod)	60-120 seconds

This is why the job is asynchronous. The teacher submits and leaves; results appear later. Design the UX around that from day one — do not build a progress bar the teacher is expected to watch.

3. Tech stack

Layer	Choice	Why
Frontend	Next.js 14 (App Router), TypeScript, Tailwind CSS	PWA installable on teacher phones without app store fees
Backend API	FastAPI (Python 3.11)	The ML libraries are Python; splitting languages adds a boundary for no gain
Job queue	Redis + RQ	Simpler than Celery, adequate here
ML runtime	InsightFace (SCRFD detector, ArcFace r100 embedder) via ONNX Runtime	Best open-source accuracy, permissively licensed, no training required
Video	ffmpeg (via subprocess), OpenCV for frame handling	Standard
Clustering	scikit-learn DBSCAN	Handles unknown cluster count, unlike k-means
Database	PostgreSQL (Supabase)	pgvector available later if the gallery grows
Object storage	Supabase Storage or any S3-compatible	
Deployment	Frontend on Vercel; API and worker on a CPU VPS (Hetzner/DigitalOcean)	Vercel cannot run the worker; a 2 vCPU VPS is roughly 500-900 rupees a month

Note that free-tier hosting ends at the worker. Video processing needs a machine you control. Budget approximately 700 rupees a month, or use Google Colab during the research phases (0-7) and only rent a VPS at Phase 9.

4. The global prompt

Paste this at the start of **every** phase session. It is the contract that keeps independently generated phases compatible.

```
You are working on a single project called "Attend". Read this entire brief
before writing any code. It applies to every task I give you in this project.

=== WHAT THE PROJECT IS ===

Attend is a classroom attendance system. A teacher records a ~60 second 4K video
```

panning across a classroom of up to 120 students and uploads it. The backend extracts frames, detects faces, embeds them with ArcFace, clusters the embeddings into per-person identities, matches those identities against a gallery of enrolled students, and produces a draft attendance list. The teacher reviews the uncertain cases and commits the record.

It is NOT a real-time system. Processing is asynchronous and takes minutes. Never design a UI or API that assumes the teacher is waiting for the result.

=== TECH STACK (do not substitute) ===

Frontend: Next.js 14 App Router, TypeScript (strict), Tailwind CSS
Backend: FastAPI, Python 3.11, Pydantic v2
Worker: RQ (Redis Queue) – NOT Celery
ML: insightface, onnxruntime, opencv-python-headless, numpy, scikit-learn
Video: ffmpeg via subprocess – NOT moviepy
Database: PostgreSQL via SQLAlchemy 2.0 (async) + Alembic migrations
Storage: S3-compatible via boto3
Testing: pytest, pytest-asyncio
Lint: ruff (Python), eslint + prettier (TypeScript)

=== REPOSITORY LAYOUT (use exactly this) ===

```
attend/  
  apps/  
    web/                Next.js frontend  
  services/  
    api/  
      app/  
        main.py          FastAPI endpoint  
        config.py        Pydantic Settings, all env vars  
        db/  
          models.py      SQLAlchemy models  
          session.py  
        routers/  
          schemas/       Pydantic request/response models  
          services/      business logic, no FastAPI imports here  
    alembic/  
    tests/  
  worker/  
    pipeline/  
      extract.py         frame extraction  
      detect.py          face detection  
      quality.py         quality gating  
      align.py           face alignment  
      embed.py           embedding  
      cluster.py         clustering  
      match.py           gallery matching  
      run.py             orchestrates the above, the job endpoint  
    tests/  
  packages/  
    contracts/           shared JSON schemas + generated TS types  
  eval/  
    datasets/            ground truth annotations  
    scripts/             evaluation harness  
  docker-compose.yml  
  .env.example
```

=== NON-NEGOTIABLE RULES ===

1. Every pipeline stage is a PURE FUNCTION with typed input and typed output, defined in its own module, and independently testable without a database, without network, and without FastAPI. The orchestrator in run.py is the only place that knows about the database.
2. Every stage writes its intermediate artifacts to disk under a per-job directory: /data/jobs/{job_id}/. Stages must be re-runnable from the previous stage's artifacts without redoing earlier work. This is essential – you will re-run the matching stage hundreds of times during threshold calibration and you must not re-extract frames each time.
3. All tunable numbers (thresholds, frame rate, minimum face size, DBSCAN parameters) live in ONE file: services/worker/pipeline/params.py, as a single frozen dataclass named PipelineParams with documented defaults. Never inline a magic number anywhere else in the pipeline. Every job record stores the exact PipelineParams used, serialised to JSON.
4. Never delete or overwrite an attendance record. Corrections create a new row with a reference to the one being corrected. The attendance table is append-only.
5. Face embeddings are biometric data. Store them in a dedicated table, never in logs, never in API responses, never in error messages. Every table holding biometric data has a created_at and a retention_expires_at column.
6. All timestamps are timezone-aware UTC in the database. Convert at the presentation layer only.
7. Every API endpoint returns a typed Pydantic model. No bare dicts. No HTTPException with a plain string – use a structured error model with a machine-readable code field.
8. Type hints on every Python function signature. TypeScript strict mode with no

```
'any'.

=== CODE STYLE ===

- Prefer explicit over clever. This code will be read by someone learning.
- Comment WHY, never WHAT. "# ArcFace expects BGR, not RGB" is useful.
  "# loop over frames" is noise.
- Functions under 50 lines. If longer, extract.
- No global mutable state.
- Fail loudly. Never silently swallow an exception in the pipeline – log the
  job_id, the stage, and re-raise. A job that fails visibly is far better than a
  job that quietly marks students absent.

=== WHEN YOU ARE UNSURE ===

Do not guess and do not invent requirements. Stop and ask me a specific question.
If you make an assumption because you cannot proceed without one, state it
explicitly at the top of your response under a heading "ASSUMPTIONS I MADE".

=== WHAT TO PRODUCE EACH TIME ===

1. The complete files, with full paths, ready to save. No elisions, no
   "... rest of the file unchanged".
2. Any new dependencies, with exact version pins.
3. Any database migration needed.
4. The tests for what you built.
5. A short "HOW TO VERIFY" section with the exact commands I should run.

Confirm you have read and understood this brief, then wait for the phase task.
```

5. The UI prompt

Supply this alongside the global prompt for Phases 0, 1, 8 and 9 only.

```
=== UI CONTRACT FOR "ATTEND" ===

Read this together with the global project brief.

WHO USES THIS

Primary user: a college teacher, aged 30-60, moderate phone literacy, standing
in a classroom, often in a hurry, frequently on a bad network. They will use this
between classes with one hand.

Secondary user: department office staff, on a desktop, generating monthly reports.

Design for the teacher first. Every screen must be usable one-handed on a
360px-wide phone.

CORE PRINCIPLES

1. The teacher's job is to CONFIRM, not to enter. The system proposes; the human
   approves. Never present an empty form where a pre-filled draft is possible.

2. Nothing blocks on the network. Every screen must render and function from
   local state. Uploads and syncs happen in the background with clear status.
   The word "offline" should never appear as an error – it is a normal condition.

3. Show uncertainty honestly. When the system is unsure about a student, say so
   plainly and make confirming it a single tap. Never present a guess as a fact.

4. Destructive and irreversible actions require a distinct confirmation step.
   Committing attendance is irreversible and must feel like it.

VISUAL SYSTEM

Framework: Tailwind CSS. No component library – write the components.

Colour: a neutral gray scale for 95 percent of the interface. Exactly three
semantic colours, used only for meaning, never decoration:
  - green  : confirmed present
  - amber  : needs teacher review
  - red    : marked absent
Do not introduce a brand colour, gradients, or shadows beyond a single subtle
elevation for cards.

Typography: system font stack. Three sizes only – 14px body, 16px emphasis,
20px screen title. Two weights only – 400 and 500. Never uppercase text.

Spacing: a 4px scale. Touch targets minimum 44 x 44 px, no exceptions.

Density: student review grids are 3 columns on phone, 6 on desktop. Face crop
fills the cell as a square; name and roll number sit below it in 11px.

Corners: 8px on cards and controls. Never fully rounded except avatars.

WRITING
```

Sentence case everywhere. No exclamation marks. No "please". No "successfully" — the confirmation state is the confirmation. Errors say what happened and what to do next, in one sentence, with no error codes shown to the teacher.

Label things in the teacher's language, not the system's. Say "needs your check", not "below confidence threshold". Say "still uploading", not "job queued".

SCREENS TO BUILD (across phases)

1. Sign in
2. Today — list of the teacher's classes for today, each with status
3. Record — camera guidance overlay and capture instructions
4. Upload status — background progress, dismissible
5. Review — the draft roster with confident/uncertain/absent bands
6. Confirm and commit — summary with irreversibility warning
7. Class history — past sessions, read only
8. Reports (desktop) — defaulter lists and monthly export

ACCESSIBILITY

Every interactive element reachable by keyboard. Every image has alt text. Colour is never the only signal — pair each state colour with an icon and a text label, because a teacher may be colour-blind and because the screen may be in sunlight.

6. The phases

Phase	Name	Focus	Rough effort at 15 hrs/week
0	Foundations and consent	Repo, schema, ethics	1 week
1	Enrollment and gallery	Building the reference set	1 week
2	Upload and orchestration	Getting video in, jobs running	1 week
3	Frames and detection	Finding faces	1 week
4	Quality gate and alignment	Discarding what will poison results	1 week
5	Embedding and clustering	Turning crops into identities	2 weeks
6	Matching and decisions	Identities to names	1 week
7	Evaluation harness	Measuring the truth	2 weeks
8	Review interface	Human in the loop	2 weeks
9	Reports and deployment	Making it usable	2 weeks
10	Continuous quality assurance	Keeping it working	1 week

Total: roughly 15 weeks at 15 hours per week. Phases 5 and 7 are where the real learning is; do not rush them.

Phase 10 can be started early. In particular, the golden-dataset regression suite is worth building as soon as Phase 7 has produced its first labelled sessions — every change you make afterwards is safer for it existing.

Phase 0 — Foundations and consent

What it does

Establishes the repository skeleton, the database schema, the configuration system, and — critically — the consent and ethics groundwork before a single face is captured.

You are about to record 90 classmates and compute biometric identifiers from their faces. Doing this without written, informed, revocable consent is wrong, and in an academic context it will also get your project shut down. Handle it first, properly, and it becomes a strength of the project rather than a liability.

The database schema is designed here in full even though most tables go unused until later phases, because retrofitting a schema across nine phases is far more painful than designing it once.

Integration contract

- Consumes: nothing.
- Produces: a running dev environment (`docker-compose up` gives you Postgres, Redis, API, worker); a migrated schema; a consent record for every enrolled student.

Definition of done

`docker-compose up` starts cleanly. `alembic upgrade head` applies. A health endpoint returns 200. You have a signed consent form template and your project supervisor has seen it.

Prompt

PHASE 0 – FOUNDATIONS AND CONSENT

Build the project skeleton for Attend as specified in the global brief.

DELIVERABLES

1. Repository structure exactly as laid out in the global brief, with every directory present and a placeholder `__init__.py` or index file where needed.
2. `docker-compose.yml` bringing up four services: postgres 16, redis 7, the FastAPI api service, and the RQ worker service. Include healthchecks. The api and worker must share a volume mounted at `/data` for job artifacts.
3. `services/api/app/config.py` – Pydantic Settings class reading from environment. Include: `DATABASE_URL`, `REDIS_URL`, `S3_ENDPOINT`, `S3_BUCKET`, `S3_ACCESS_KEY`, `S3_SECRET_KEY`, `JOB_DATA_DIR`, `BIOMETRIC_RETENTION_DAYS` (default 180), `ENVIRONMENT`. Provide a matching `.env.example`.
4. SQLAlchemy 2.0 async models in `services/api/app/db/models.py` for this schema:

```
institution      id, name, created_at
department       id, institution_id, name, code
course           id, department_id, code, title, semester
student          id, department_id, roll_number (unique per department),
                 full_name, admission_year, is_active
teacher          id, department_id, full_name, email (unique), password_hash
enrollment       id, student_id, course_id (which students are in which course)
class_session    id, course_id, teacher_id, scheduled_at, room, status
                 status is one of: scheduled, recording, processing,
                 awaiting_review, committed, failed

consent          id, student_id, granted_at, revoked_at (nullable),
                 consent_version, scope (text), evidence_uri
gallery_photo    id, student_id, storage_uri, captured_at, quality_score
gallery_embedding id, student_id, vector (bytea, 512 float32),
                 source_photo_id, model_version, created_at,
                 retention_expires_at

video_upload     id, class_session_id, storage_uri, duration_seconds,
                 width, height, fps, bytes, uploaded_at
processing_job   id, class_session_id, video_upload_id, state, stage,
                 params_json, error_text, started_at, finished_at
                 state is one of: queued, running, succeeded, failed
detected_cluster id, processing_job_id, representative_vector (bytea),
                 crop_count, mean_quality, best_crop_uri
cluster_match    id, cluster_id, student_id (nullable), similarity,
                 runner_up_similarity, decision
                 decision is one of: confident, uncertain, unmatched

attendance_record id, class_session_id, student_id, status, source,
                 confidence (nullable), decided_by_teacher_id (nullable),
                 supersedes_id (nullable), created_at
                 status is one of: present, absent
                 source is one of: auto, teacher_confirmed, teacher_override

audit_log        id, actor_type, actor_id, action, subject_type, subject_id,
                 payload_json, created_at
```

Enforce: `attendance_record` is append-only (no update or delete permitted at the service layer – add a comment documenting this constraint). Add indexes on every foreign key and on `(class_session_id, student_id)` for `attendance_record`.

5. Alembic configured for async, with the initial migration generated.
6. FastAPI app with `/health` returning service name, version, and database and redis connectivity status. Structured JSON logging with a request id.
7. A consent document at `docs/CONSENT.md` written in plain English suitable for a college student to read and sign. It must state: what is recorded (video of the classroom), what is derived (a mathematical face descriptor, not a stored photograph beyond the enrollment set), how long it is kept, that it will not be shared outside the project, how to revoke consent, and who to contact. Also provide a Tamil translation placeholder section.
8. A `CONSENT ENFORCEMENT` rule in the service layer: a helper function `assert_consent_valid(student_id)` that raises if there is no consent row, or if `revoked_at` is set. Document that this must gate every enrollment and every match operation.
9. pytest set up with a test database fixture, and one test per model asserting it can be created and queried.

HOW TO VERIFY should include the exact docker compose and alembic commands.

Phase 1 — Enrollment and gallery

What it does

Builds the reference set the whole system matches against. Quality here determines your ceiling.

The instinct is to use existing admission photographs. Resist it. A three-year-old, badly-lit, single-angle admission photo is the most common cause of poor matching accuracy in student projects. The domain gap between a formal frontal portrait and a classroom pan is large.

Instead, enroll from a short self-recorded video: each student records five seconds turning their head slowly left to right. You then run the same detection, quality-gate and embedding code over it and keep the best five to eight crops. This gives you multi-angle, current, same-camera-domain references, and it reuses the pipeline you are building anyway.

Integration contract

- Consumes: consent records from Phase 0.
- Produces: 5-8 `gallery_embedding` rows per student, each a 512-float vector, plus a per-student mean vector cached for fast matching.

Definition of done

Every student in your test class has at least five gallery embeddings from at least three distinct head poses. A sanity script shows that within-student similarity is high (typically above 0.6) and between-student similarity is low (typically below 0.3). If those two distributions overlap, your enrollment is bad and nothing downstream will work.

Prompt

PHASE 1 – ENROLLMENT AND GALLERY CONSTRUCTION

Build the student enrollment pipeline.

CONTEXT

Students enroll by recording a 5-second video of their own face while slowly turning their head from left profile to right profile. From this we extract multiple high-quality, multi-pose reference embeddings.

DELIVERABLES

1. `services/worker/pipeline/params.py` – the single frozen dataclass `PipelineParams` holding every tunable in the project. Initialise with these documented defaults (they will be calibrated in Phase 7, so every one needs a comment explaining what it controls and which direction to move it):

```
sample_fps: float = 4.0
detector_score_min: float = 0.60
min_face_px: int = 50
max_abs_yaw_deg: float = 35.0
max_abs_pitch_deg: float = 25.0
blur_laplacian_min: float = 90.0
embed_input_size: int = 112
cluster_eps: float = 0.42          # DBSCAN cosine distance
cluster_min_samples: int = 3
match_threshold: float = 0.38     # cosine similarity, ArcFace r100
match_margin_min: float = 0.05    # top1 minus top2
uncertain_band: float = 0.08      # below threshold by this much -> review
gallery_min_embeddings: int = 5
```
2. `services/worker/pipeline/embed.py` – loads InsightFace ArcFace r100 (`buffalo_l`) via `onnxruntime`, exposes:

```
load_model(model_dir: Path) -> EmbedModel
embed_batch(model: EmbedModel, aligned: np.ndarray) -> np.ndarray
```

Input is (N, 112, 112, 3) BGR uint8. Output is (N, 512) float32, L2-normalised. Cache the model as a module-level singleton – loading it per call will destroy performance.
3. An enrollment job that given a `student_id` and a video URI:
 - extracts frames at 6 fps
 - detects faces (use insightface SCRFD; put the wrapper in `detect.py`)
 - keeps only the largest face per frame (an enrollment video has one subject; if a frame contains two faces, discard that frame entirely and log it)
 - applies the quality gate from `params`
 - estimates yaw from the 5 landmarks and buckets frames into three pose groups: left (yaw < -15), frontal (-15 to 15), right (> 15)
 - selects the top N by quality score from each bucket so the final set spans poses rather than being 8 near-identical frontal frames
 - embeds them, writes `gallery_photo` and `gallery_embedding` rows
 - sets `retention_expires_at` to now + `BIOMETRIC_RETENTION_DAYS`
 - calls `assert_consent_valid(student_id)` FIRST and aborts if it fails
4. A cached per-student mean vector: add a materialised table or a cached column holding the L2-normalised mean of that student's embeddings, recomputed on enrollment change. This is what matching will use.
5. API endpoints:

```
POST /students/{id}/enrollment  accepts video upload, queues job
GET  /students/{id}/enrollment  returns status and pose coverage
```

```
DELETE /students/{id}/enrollment    deletes all biometric data for that
                                     student and writes an audit_log row
```

6. eval/scripts/gallery_sanity.py – a script that computes and prints:
 - within-student cosine similarity distribution (mean, p5, p50, p95)
 - between-student cosine similarity distribution (same stats)
 - the overlap between the two distributions
 - a list of any student whose within-student mean is below 0.55, flagged for re-enrollmentPrint it as a text histogram. This script is how I will know whether my enrollment data is usable before I build anything else.
7. Tests: embed_batch output shape and normalisation; pose bucketing on synthetic landmark inputs; consent gate rejects an unconsented student.

Phase 2 — Upload and orchestration

What it does

Gets a large video file from a phone on a bad network into object storage reliably, and turns it into a queued job. Unglamorous, and the most common place student projects break in real deployment.

The key design decision: resumable chunked upload. A 300 MB upload over college wifi will fail partway. If a failure means starting over, the system is unusable in practice regardless of how good the recognition is.

Integration contract

- Consumes: a `class_session` from Phase 0.
- Produces: a `video_upload` row, a stored video object, and a `processing_job` in state `queued` with the exact `PipelineParams` serialised into it.

Definition of done

You can upload a 300 MB file from a phone, kill the network halfway, restore it, and the upload resumes and completes. The job appears in Redis. Killing the worker mid-job and restarting it does not corrupt state.

Prompt

PHASE 2 – VIDEO UPLOAD AND JOB ORCHESTRATION

Build reliable large-file upload and the job orchestration skeleton.

DELIVERABLES

1. Resumable chunked upload:

```
POST /uploads                create session, returns upload_id, chunk size
PUT  /uploads/{id}/chunks/{n} upload one chunk, idempotent
GET  /uploads/{id}           returns which chunk indices are received
POST /uploads/{id}/complete  assemble, validate, finalise
```

Chunk size 5 MB. Store chunks in object storage, assemble server-side on completion. PUT of an already-received chunk must return 200 and change nothing – the client will retry.

2. On completion, probe the file with ffprobe and record duration, width, height, fps, codec, bytes into `video_upload`. REJECT the upload with a clear structured error if:
 - duration is under 20 seconds or over 5 minutes
 - the shorter dimension is under 1080 px
 - the file is not a video ffprobe can read

The rejection message shown to the teacher must be plain language, for example "This video is too short. Record at least 30 seconds, panning slowly across the room."

- 2b. PRE-FLIGHT QUALITY CHECK – `services/worker/pipeline/preflight.py`

```
preflight_check(video_path, expected_students, params) -> PreflightResult
```

This runs SYNCHRONOUSLY on upload completion and must finish in under 30 seconds. It samples 15-20 evenly spaced frames and evaluates:

- median Laplacian variance across sampled frames (overall sharpness)
- backlight ratio: mean luminance of the upper third versus mean luminance of detected face regions; a large positive ratio means windows behind the students
- pan detection: run a cheap detection pass on the sampled frames and track the mean x-position of detections over time. A genuine pan produces a monotonic sweep. A static camera produces a flat line.
- pan speed: mean absolute per-frame change in that x-position, scaled by frame interval
- face yield: mean detections per sampled frame versus expected_students
- coverage: the span between the minimum and maximum detection x-position across the whole video, as a fraction of frame width

Return a structured result with a status of pass, warn or fail, plus a list of individual check results each carrying a machine code, a severity, and a

plain-language message for the teacher. Every threshold used here is a field on PipelineParams.

A fail must be surfaced to the teacher IMMEDIATELY, while they are still in the classroom, with a message telling them what to change and a one-tap "record again" action. A warn proceeds to processing but is stored on the job and shown alongside the results.

Rationale to keep in mind: discovering a bad video twenty minutes later is useless because the class has ended. Discovering it in twenty seconds means the teacher simply re-shoots. This check is worth more to real-world reliability than several points of model accuracy.

3. Job orchestration in services/worker/pipeline/run.py:
 - a single RQ job function process_session(job_id)
 - it reads the processing_job row, deserialises PipelineParams from params_json, and runs stages in order
 - after each stage it writes the stage name to processing_job.stage and persists that stage's artifacts under /data/jobs/{job_id}/{stage}/
 - if an artifact directory for a stage already exists and is complete, the stage is SKIPPED and its artifacts loaded from disk
 - on exception: log job_id, stage, traceback; set state=failed and error_text; re-raise so RQ records the failureFor this phase, implement the orchestrator with the real stage sequence but stub each stage as a function that logs and writes an empty manifest. Later phases fill them in one at a time.
4. A stage artifact manifest convention: each stage directory contains a manifest.json with {stage, params_hash, completed_at, item_count}. A stage is considered complete only if manifest.json exists and params_hash matches the current params. Changing a parameter must invalidate that stage and everything after it, but not stages before it. Implement this invalidation logic and test it – it is what makes Phase 7 calibration feasible.
5. API: POST /sessions/{id}/process to enqueue, GET /jobs/{id} for status.
6. Frontend upload component with background upload, progress, automatic retry with exponential backoff, and resume on app reopen. It must survive the browser tab being backgrounded.
7. Tests: chunk idempotency; resume after simulated failure; params_hash invalidation cascade; orchestrator skips completed stages.

Phase 3 — Frames and detection

What it does

Turns the video into a set of face detections. Two decisions matter here.

First, sampling rate. At 30 fps a 60-second video is 1800 frames — far more than needed, since consecutive frames are near-identical. Sampling at 4 fps gives roughly 240 frames, which is ample coverage while cutting compute by seven-eighths.

Second, detection at 4K. SCRFD is typically run at 640px input. Feeding it a downscaled 4K frame makes back-row faces vanish. The fix is tiled detection: split each 4K frame into overlapping tiles, detect on each tile at native resolution, then merge detections with non-maximum suppression. This is the difference between finding 40 faces and finding 88.

Integration contract

- Consumes: the stored video.
- Produces: frames/ (sampled JPEGs) and detections/detections.parquet — one row per detected face with frame index, bounding box, 5 landmarks, detector score.

Definition of done

On your real classroom video, the total detection count across all frames is at least 30 times the number of students present, and visual spot checks on 10 random frames show back-row faces being found.

Prompt

PHASE 3 – FRAME EXTRACTION AND FACE DETECTION

DELIVERABLES

1. services/worker/pipeline/extract.py

```
extract_frames(video_path: Path, out_dir: Path, params: PipelineParams)
-> FrameManifest
```

Uses ffmpeg via subprocess to sample at params.sample_fps, writing full resolution JPEGs at quality 95 named frame_%05d.jpg. Do NOT resize. Returns a manifest with frame count, source timestamp per frame, and dimensions. Use a single ffmpeg invocation with the fps filter – do not loop in Python.
2. services/worker/pipeline/detect.py

TILED DETECTION – this is the critical part of this phase.

```
detect_faces(frame_paths, out_dir, params) -> DetectionSet
```

For each frame:

- a. If the frame's longer side is over 2000 px, split it into a grid of overlapping tiles. Tile size 1280 px, overlap 256 px. Compute the grid so tiles cover the full frame.
- b. Run SCRFD (insightface, det_10g model) on each tile at its native size.
- c. Map each tile-local detection back to frame coordinates.
- d. Merge across tiles with non-maximum suppression at IoU 0.4.
- e. Also run detection once on the whole frame downsampled to 1280, and merge those results in too – this catches large front-row faces that a tile may have cut in half.

Output one row per detection into a parquet file with columns:

```
frame_index, frame_timestamp_s, det_id, x1, y1, x2, y2, score,  
lmk_x1..lmk_x5, lmk_y1..lmk_y5, face_width_px, tile_origin_x, tile_origin_y
```

Store nothing else. No crops yet – cropping happens in Phase 4.

3. Parallelism: process frames with a multiprocessing Pool sized to `min(cpu_count, 4)`. The ONNX session must be created once per worker process, not per frame. Getting this wrong makes the stage 20x slower.
4. A debug utility `eval/scripts/draw_detections.py` that renders boxes onto N sampled frames and writes them to a directory, so I can visually verify that back-row faces are being detected. Include the `face_width_px` as a label on each box.
5. Log at the end of the stage: total frames, total detections, detections per frame (mean/min/max), and a histogram of `face_width_px` in 10px buckets. That histogram is the single most diagnostic output of this phase – if the mass is below 50 px, the capture was wrong and no downstream work will save it.
6. Tests: tile grid covers the frame with correct overlap; NMS merges duplicate detections across a tile boundary; coordinate mapping from tile-local to frame is exact.

Phase 4 — Quality gate and alignment

What it does

Discards crops that will produce wrong answers, and geometrically normalises the survivors.

This phase is counterintuitive and is where most projects lose accuracy. The instinct is that more data is better, so keep every detection. The opposite is true: a blurred, tiny, or extreme-profile face does not produce a weak signal — it produces a confidently wrong embedding that can pull a cluster toward the wrong identity. Aggressive filtering raises accuracy.

Expect to discard 70-80 percent of detections here. That is correct and healthy.

Integration contract

- Consumes: `detections.parquet` and the frame JPEGs.
- Produces: `aligned/` (112x112 BGR crops as a single memory-mapped numpy array) and `quality.parquet` with per-crop quality scores and the reason any crop was rejected.

Definition of done

Rejection reasons are logged with counts. Manually inspecting 30 accepted and 30 rejected crops confirms the gate is making sensible calls. Roughly 3000-6000 crops survive from a 90-second video.

Prompt

PHASE 4 – QUALITY GATING AND FACE ALIGNMENT

DELIVERABLES

1. `services/worker/pipeline/quality.py`

```
score_detections(detections_df, frame_dir, params) -> QualityResult
```

For every detection compute:

- `face_width_px` (already present)
- `blur`: variance of the Laplacian over the grayscale crop. Compute it on the crop resized to a fixed 128x128 first, otherwise the metric scales with face size and is not comparable across rows.
- `brightness`: mean luminance; flag if under 40 or over 215
- `yaw` and `pitch` estimated from the 5 landmarks. Document the geometry you use. A workable approach: `yaw` from the horizontal offset of the nose tip relative to the midpoint of the two eyes, normalised by inter-ocular distance; `pitch` from the vertical offset of the nose relative to the eye-mouth midpoint.
- a composite quality score in 0..1 combining normalised face size, blur and frontality. Document the weighting; make the weights fields on `PipelineParams` so they can be tuned in Phase 7.

Then apply the gate. A detection is REJECTED if any of:

```
score < detector_score_min
face_width_px < min_face_px
blur < blur_laplacian_min
abs(yaw) > max_abs_yaw_deg
abs(pitch) > max_abs_pitch_deg
brightness outside range
```

Write quality.parquet with every input row plus the computed metrics, a boolean accepted column, and a reject_reason string. Keep the rejected rows – do not drop them. Phase 7 needs to analyse what was thrown away.

2. services/worker/pipeline/align.py

```
align_crops(accepted_df, frame_dir, out_dir, params) -> AlignedSet
```

Standard ArcFace 5-point alignment: similarity transform mapping the detected 5 landmarks onto the canonical ArcFace reference landmark positions for a 112x112 output. Use cv2.estimateAffinePartial2D with method LMEDS, then cv2.warpAffine.

Hardcode the canonical reference points as a module constant with a comment citing that they are the standard ArcFace/InsightFace reference. Getting these wrong silently degrades every embedding, so add a test that aligns a synthetic face with known landmarks and asserts the eyes land within 2 px of the expected canonical positions.

Write output as a single numpy memmap file aligned.npy of shape (N, 112, 112, 3) uint8 BGR, plus aligned_index.parquet mapping row index back to det_id. A memmap, not individual JPEGs – you will load this array repeatedly in later phases and JPEG decode overhead will dominate.

3. Stage summary log: input detections, accepted count, rejected count broken down by reason, and the accepted-crop quality score distribution.
4. eval/scripts/contact_sheet.py – writes a grid image of N randomly sampled accepted crops and N rejected crops with their reject reason labelled, so I can eyeball whether the gate is calibrated sensibly.
5. Tests: alignment canonical position test as described; each reject rule fires on a crafted input; blur metric is scale-invariant across two sizes of the same face.

Phase 5 — Embedding and clustering

What it does

This is the intellectual core of the project. It converts several thousand anonymous face crops into roughly ninety identities, without knowing in advance how many people are in the room or who they are.

Each aligned crop becomes a 512-dimensional vector where distance corresponds to identity difference. Crops of the same person across the pan land near each other; different people land far apart. Clustering recovers the groups.

The payoff, as described in Section 2.2, is noise averaging. One crop of a back-row student might embed poorly. Forty crops of that student, averaged with quality weighting, produce a representative vector far cleaner than any individual crop. This is where the accuracy comes from — not from the model, from the aggregation.

DBSCAN rather than k-means, because you do not know the number of people present. That is precisely the thing you are trying to determine.

Integration contract

- Consumes: `aligned.npy` and `quality.parquet`.
- Produces: `embeddings.npy` of shape (N, 512) float32 L2-normalised, and `clusters.parquet` assigning each crop to a cluster id (or -1 for noise), plus per-cluster representative vectors.

Definition of done

Cluster count is within roughly 20 percent of the true number of people present. Manual inspection of five clusters shows each contains one person. You have identified and can describe your failure modes — over-splitting (one person in three clusters) and merging (two people in one cluster).

Prompt

PHASE 5 — EMBEDDING AND IDENTITY CLUSTERING

This is the core algorithmic phase. Read it fully before starting.

DELIVERABLES

1. Complete services/worker/pipeline/embed.py for the batch case:

```
embed_aligned(aligned_memmap, out_dir, params) -> EmbeddingSet
```

- Load ArcFace r100 (buffalo_l recognition model) once as a process singleton.
- Process in batches of 64.
- Input must be BGR uint8 (112,112,3); insightface expects BGR – add a comment asserting this because it is the single most common silent bug here.
- Output float32 (N,512), L2-normalised. Write to embeddings.npy as memmap.
- Log throughput in crops per second.

2. services/worker/pipeline/cluster.py

```
cluster_embeddings(embeddings, quality_df, params) -> ClusterResult
```

Algorithm:

- DBSCAN with metric='cosine', eps=params.cluster_eps, min_samples=params.cluster_min_samples.
- Points labelled -1 are noise. Do NOT discard them – record them; a genuine back-row student may appear as noise, and Phase 7 needs to know how often that happens.
- For each cluster, compute a representative vector as the QUALITY-WEIGHTED mean of its member embeddings, then L2-normalise. Weight by the composite quality score from Phase 4. Document why weighting beats a plain mean.
- Compute per-cluster diagnostics: member count, mean quality, intra-cluster mean cosine similarity (cluster tightness), and the frame index range its members span.
- Select a best_crop per cluster – the highest quality member – and save it as a JPEG for later display in the teacher review UI.

Write clusters.parquet: det_id, cluster_id, distance_to_representative.

Write cluster_summary.parquet: cluster_id, member_count, mean_quality, tightness, first_frame, last_frame, best_crop_uri.

3. TEMPORAL COHERENCE POST-PASS – implement this and make it toggleable via a params flag, because it is a genuine improvement I want to be able to measure on and off:

Because the camera pans, a given person is visible only during a contiguous window of frames. Therefore:

- If two clusters have representative vectors closer than params.cluster_eps * 1.3 AND their frame ranges overlap substantially, they are likely the same person over-split by DBSCAN. Merge them.
- If a single cluster's members span a frame range wider than roughly 60 percent of the total video AND the cluster tightness is poor, it is likely two different people merged. Flag it for splitting via a second DBSCAN pass at tighter eps restricted to that cluster's members.

Log every merge and split decision.

4. eval/scripts/cluster_report.py – for a given job, produce:

- number of clusters found, number of noise points
- a contact sheet image per cluster: a grid of up to 12 member crops, so I can visually verify a cluster is one person
- the distribution of cluster sizes
- the distribution of cluster tightness

The per-cluster contact sheets are the main learning artifact of this phase.

Looking at them is how you will understand what your system is actually doing.

5. A parameter sweep helper eval/scripts/sweep_cluster.py that re-runs ONLY the clustering stage across a grid of cluster_eps values from 0.30 to 0.55 in steps of 0.02 and min_samples in {2,3,4,5}, reporting cluster count and, if ground truth is available, purity. It must reuse cached embeddings and never re-run detection or embedding – this is why the stage caching from Phase 2 exists.

6. Tests: cluster_embeddings on synthetic data with three well-separated Gaussian blobs returns exactly three clusters; quality weighting shifts the representative toward high-quality members; the merge rule fires on constructed overlapping clusters.

Phase 6 — Matching and decisions

What it does

Assigns names to clusters, and — more importantly — decides when *not* to.

The three-band decision is the design that makes this system safe to deploy. Rather than forcing a binary present/absent call on every student, classify each cluster as confidently matched, uncertain, or unmatched, and route only the uncertain ones to the teacher. A system that is 92 percent accurate and honest about the other 8 percent is usable. A system that is 92 percent accurate and silent about it is not.

Note also the one-to-one constraint: one physical person cannot be two students. Greedy top-1 matching violates this. Use the Hungarian algorithm for an optimal global assignment.

Integration contract

- Consumes: `cluster_summary.parquet` with representative vectors; the student gallery for the course.
- Produces: `cluster_match` rows and a draft attendance list.

Definition of done

Given a class session, the system produces a draft roster with confident, uncertain and absent bands, and no student appears in two bands.

Prompt

PHASE 6 – GALLERY MATCHING AND THREE-BAND DECISIONS

DELIVERABLES

1. services/worker/pipeline/match.py

```
match_clusters(cluster_reps, gallery, params) -> MatchResult

a. Build the gallery matrix: for each student enrolled in this course, their
   cached mean embedding, L2-normalised, shape (S, 512). Only students in the
   course's enrollment table – never the whole institution.
b. Compute the full cosine similarity matrix (C, S) between cluster
   representatives and gallery vectors. Since both are L2-normalised this is
   a single matrix multiply. Do not loop.
c. ONE-TO-ONE ASSIGNMENT: use scipy.optimize.linear_sum_assignment (Hungarian)
   on the negated similarity matrix to find the globally optimal assignment.
   Explain in a comment why greedy top-1 is wrong here: two clusters of two
   different students could both have their highest similarity to the same
   gallery entry, and greedy would assign one of them incorrectly rather than
   finding the arrangement that maximises total similarity.
d. For each assigned pair record: similarity, and runner_up_similarity (the
   best similarity to any OTHER gallery entry).
```

2. Three-band decision rule:

```
margin = similarity - runner_up_similarity

CONFIDENT if similarity >= match_threshold AND margin >= match_margin_min
UNCERTAIN if similarity >= (match_threshold - uncertain_band)
           AND not confident
UNMATCHED otherwise
```

Write cluster_match rows with the decision.

3. Draft roster construction:

- Every student with a CONFIDENT match -> proposed present, source auto
- Every student with an UNCERTAIN match -> goes to the review queue, showing the cluster's best crop next to the student's enrollment photo
- Every student with no cluster assigned -> proposed absent, but ALSO placed in the review queue at lower priority, because a genuinely present student who was never detected is the failure mode that most damages trust
- Every cluster that matched no student (UNMATCHED with no assignment) -> surfaced as "unrecognised person" with its best crop, for the teacher to either ignore or flag

IMPORTANT: at this phase, do NOT write attendance_record rows. This produces a DRAFT only, stored against the processing job. Attendance is committed by a human in Phase 8. Set class_session.status = awaiting_review.

4. SESSION SUMMARY – compute and store on the draft a summary object:

total_enrolled	count of active students enrolled in this course
proposed_present	confident matches
needs_review	uncertain matches
proposed_absent	students with no cluster assigned
unrecognised_clusters	clusters matching no student
coverage_percent	(proposed_present + needs_review) / total_enrolled
mean_confident_similarity	
session_health	a derived rating of good, fair or poor

Assert as an invariant that proposed_present + needs_review + proposed_absent == total_enrolled exactly. If it does not, fail the job loudly rather than producing a roster that silently omits a student. Add a test for this.

session_health is poor when coverage_percent is low, mean_confident_similarity is low, or the pre-flight check returned warnings. A poor rating is shown to the teacher as "the video quality was low today – please check the list carefully" so they know to apply more scrutiny rather than trusting a bad run.

- API: GET /sessions/{id}/draft returning the summary object plus the three bands. For each entry include the student name, roll number, similarity, the cluster best-crop URI where one exists, and the enrollment photo URI. Every student in every band must carry a displayable photo – for absent students that is their enrollment photo, since no crop of them exists today. Never include raw embedding vectors in any API response.
- eval/scripts/match_report.py printing, for a job: counts per band, the similarity distribution for each band, and the ten lowest-margin confident matches (these are your most likely errors – inspect them).
- Tests: Hungarian assignment beats greedy on a constructed matrix where they differ; band boundaries are exact at the threshold values; a student not enrolled in the course is never matched; the band totals invariant holds.

Phase 7 — Evaluation harness and calibration

What it does

This phase is the actual point of the project. Everything before it was plumbing; this is where you find out what you built, and it is the part that will teach you the most and that examiners and employers will care about.

You will build ground truth, define metrics, sweep thresholds, and produce an honest accuracy figure with its failure modes characterised. The difference between an engineer who says “it works, about 95 percent” and one who says “per-student recall is 0.94, precision 0.98, with recall dropping to 0.71 for rows five and beyond and to 0.83 for students wearing glasses, at an operating threshold of 0.41 chosen to hold false-present below 1 percent” is the entire difference in seniority.

Integration contract

- Consumes: labelled ground truth for a set of recorded sessions; cached pipeline artifacts.
- Produces: a metrics report, a chosen operating threshold, and a documented characterisation of failure modes.

Definition of done

You have at least eight labelled sessions. You can state precision, recall and F1 for student-level presence, with a threshold chosen deliberately rather than by default, and you can name your three largest failure modes with evidence.

Prompt

PHASE 7 — EVALUATION HARNESS AND THRESHOLD CALIBRATION

This phase produces the project's central result. Build it carefully.

DELIVERABLES

1. Ground truth format and tooling.
eval/datasets/{session_id}/truth.csv with columns:
roll_number, actually_present (bool), row_number (int, 1 = front),
seat_position (left/centre/right), wears_glasses (bool),
notes (free text)

Build eval/scripts/label_session.py – a small local tool that shows me each enrolled student and lets me mark present/absent quickly while watching the video, writing truth.csv. Labelling 90 students x 8 sessions by hand is the bottleneck of this phase; make the tool fast. Keyboard-driven, no mouse.

2. eval/scripts/evaluate.py – the core harness.

For a set of sessions and a given PipelineParams, compute:

STUDENT-LEVEL PRESENCE METRICS (the headline numbers)

- true positive: student present and system says present
- false positive: student absent and system says present (proxy hole)
- false negative: student present and system says absent (trust killer)
- precision, recall, F1, and overall accuracy
- report false positive and false negative counts separately and prominently; they have very different real-world costs

CLUSTERING QUALITY METRICS

- number of clusters vs number of people actually present
- cluster purity: fraction of clusters containing crops of exactly one person (requires spot-labelled clusters; support partial labelling)
- over-split rate and merge rate

PIPELINE YIELD

- detections per present student
- fraction of present students who produced zero accepted crops (these are unrecoverable failures – the most important single diagnostic)

STRATIFIED BREAKDOWN – compute every headline metric separately by:

- row_number (front rows vs back rows)
- seat_position
- wears_glasses
- and, if you record it, any other student attribute in truth.csv

The stratified breakdown is mandatory, not optional. An aggregate of 94 percent that is 99 percent in row 1 and 68 percent in row 6 is a completely different system from a uniform 94 percent, and only the breakdown reveals it.

3. eval/scripts/sweep_threshold.py
Re-run ONLY the match stage across match_threshold from 0.25 to 0.60 in steps of 0.01, using cached embeddings and clusters. For each value output precision, recall and F1. Produce:
 - a precision-recall curve
 - a plot of false-positive count and false-negative count vs threshold
 - a printed table

Then explicitly choose an operating point and JUSTIFY it in a written paragraph. Recommended framing: false positives break the anti-proxy claim,

false negatives break teacher trust. Decide which you are optimising for and say so. Write the chosen value into `params.py` with a comment recording the date, the dataset it was calibrated on, and the reasoning.

4. `eval/scripts/failure_gallery.py`
For every false negative and false positive, produce a page showing the student's enrollment photos, any crops that were detected for them, what cluster they landed in, and what the system decided. This is how you diagnose causes rather than guessing.
5. A written report at `docs/EVALUATION.md` containing: dataset description (sessions, students, conditions), method, the metrics table, the stratified breakdown, the threshold selection reasoning, the three largest failure modes with example images, and a limitations section. Write it as if for a technical reviewer who is sceptical.
6. An ablation study: report headline metrics with each of the following disabled, one at a time, to quantify what each contributes:
 - tiled detection off (detect on downscaled full frame only)
 - quality gating off (embed everything)
 - clustering off (match every crop individually and majority vote)
 - temporal coherence post-pass off
 - quality weighting off (plain mean representative)This table is the most interesting result the project will produce, and it is what turns a build into research.

Phase 8 — Review interface and commit

What it does

Closes the loop with a human. The system proposes; the teacher decides; the record becomes real.

Design principle: the teacher's total interaction should be under 30 seconds. If review takes longer than manual roll call, the system has no reason to exist. This means confident matches are collapsed by default, the uncertain queue is a single swipe or tap per student, and the commit is one deliberate action.

Integration contract

- Consumes: the draft roster from Phase 6.
- Produces: committed, append-only `attendance_record` rows and `audit_log` entries.

Definition of done

A teacher who has never seen the app can review and commit a 90-student session in under a minute without instruction.

Prompt

PHASE 8 – TEACHER REVIEW INTERFACE AND ATTENDANCE COMMIT

Apply the UI contract alongside the global brief.

DELIVERABLES

1. Review screen (mobile-first). Structure top to bottom:
 - a. Summary header, always visible and always showing all four numbers:

Total enrolled	90
Confirmed present	72
Need your check	11
Not found	7

Plus a live attendance percentage that updates as the teacher works, and the `session_health` indicator from Phase 6. Never show a subset of these numbers – a teacher must always be able to see that the parts sum to the whole, because that is how they trust the list.
 - b. NEEDS YOUR CHECK – the priority section, expanded by default.

One card per uncertain student showing side by side: the enrollment photo and the detected crop from today, with the student name and roll number. Two large buttons: "Yes, present" and "Not them". Support keyboard arrows on desktop and swipe on mobile.
 - c. NOT FOUND – students the system saw no evidence of. This section is the one most likely to contain an error, so it is EXPANDED by default, not collapsed.

Render it as a photo grid using each student's `ENROLLMENT` photo, with name and roll number beneath. There is no detected crop for these students by definition – that absence is precisely the point – so label the section honestly: "We did not find these students in today's video. Tap anyone who was actually here."

Tapping a student marks them present with source `teacher_override`. Default state is absent. Show the running count of how many the teacher has overridden, because a large number is a signal the capture was poor and should be surfaced in the commit summary.
 - d. CONFIRMED PRESENT – collapsed by default, showing a count. Expandable to a

- photo grid where any student can be tapped to override to absent.
- e. UNRECOGNISED PEOPLE – if any clusters matched nobody, show their crops with the label "Someone was here we could not identify" and a single action to dismiss or report. Do not make this alarming; it is usually a student with poor enrollment photos.
2. Progressive review: the teacher must be able to commit after handling only the "needs your check" section. Do not require them to expand everything.
 3. Commit flow:
 - A distinct confirmation screen restating the final counts
 - Explicit text that the record cannot be edited afterwards, only corrected with a visible correction entry
 - On commit, write attendance_record rows for EVERY enrolled student (present or absent), each with the correct source value: auto, teacher_confirmed, or teacher_override, and confidence where applicable
 - Write an audit_log entry recording the teacher, the timestamp, the counts, and the params_hash used
 - Set class_session.status = committed
 - The commit endpoint must be idempotent on a client-supplied request id, so a double tap on a bad network cannot double-commit
 4. Correction flow: a committed session can be opened read-only. Changing a student inserts a NEW attendance_record row with supersedes_id pointing at the old one. Queries for current attendance select the row with no successor. Add a database view current_attendance implementing this and use it everywhere.
 5. Offline behaviour: the review screen must load from cached draft data and accept the teacher's decisions offline, queueing the commit for when the network returns. Show the pending state honestly.
 6. Tests: idempotent commit; supersede chain resolves to the latest row; current_attendance view correctness with a three-deep correction chain; committing writes a row for every enrolled student with no omissions.

Phase 9 — Reports, privacy and deployment

What it does

Turns a working prototype into something a department could actually use, and closes out the data protection obligations you took on when you started computing face embeddings.

The reports matter more than they sound. As discussed earlier in your planning, the expensive pain in a college is not marking attendance — it is aggregating it. Match your college's existing report format exactly.

Integration contract

- Consumes: `current_attendance`.
- Produces: exports, a retention job, and a deployed system.

Definition of done

The department office can generate the monthly report they currently produce by hand, in the format they already use. Biometric retention deletion runs and is verified. The system runs on a real server.

Prompt

PHASE 9 – REPORTS, PRIVACY HARDENING AND DEPLOYMENT

DELIVERABLES

1. Reporting API and desktop screens:
 - per-student cumulative attendance percentage per course, per semester
 - defaulter list at a configurable threshold (default 75 percent) with current percentage and the number of classes needed to recover
 - per-course, per-teacher and per-session summaries
 - date range filteringAll reads go through the current_attendance view, never the raw table.
- 1b. SESSION SUMMARY TABLE – add a denormalised session_summary table written once at commit time, holding: class_session_id, total_enrolled, present_count, absent_count, attendance_percent, auto_count, teacher_confirmed_count, teacher_override_count, session_health, preflight_status, mean_confident_similarity, committed_at, committed_by.

Reports read from this rather than recomputing across the attendance table every time. Add a consistency check job that recomputes a sample of summaries from current_attendance and alerts on any mismatch.
- 1c. ABSENTEE REPORT WITH PHOTOS – a per-session view and a printable or PDF export showing every absent student as a photo card: enrollment photo, name, roll number, their current cumulative percentage for that course, and how they were marked (auto or teacher override).

This is the artifact a department actually uses. It is what gets pinned to a

noticeboard, sent to a class advisor, or attached to a parent intimidation. Sort it by cumulative percentage ascending so the students in most difficulty appear first. Include the session summary counts in the header.

Also provide a cumulative version: for a date range, every student below the threshold, with photo, current percentage, and their absence pattern shown as a simple row of marks per session so a pattern such as "absent every Friday" is visible at a glance.

2. Export to XLSX using openpyxl. The column layout must be configurable via a template definition file, because every college's affiliating university wants a different format and hardcoding one guarantees rework. Ship one example template and document how to add another.
3. Privacy and retention hardening:
 - A scheduled RQ job running daily that deletes gallery_embedding and detected_cluster rows past retention_expires_at, deletes the corresponding stored crops and source videos, and writes an audit_log entry with counts. Attendance records themselves are NOT deleted – they are ordinary academic records, not biometric data.
 - An endpoint for a student to request deletion of all their biometric data, which revokes consent, deletes embeddings and photos, and leaves existing attendance records intact.
 - Verify that no API response, log line or error message anywhere contains a raw embedding vector. Add a test that asserts this against every endpoint.
 - Encrypt the embedding column at rest, or document explicitly why you have not and what the risk is.
 - Write docs/DATA_PROTECTION.md covering: what personal and biometric data is collected, the lawful basis and consent mechanism, retention periods, access controls, the deletion process, and breach response. Reference India's DPDP Act 2023 and the DPDP Rules 2025 obligations for biometric data and for data principals under 18.
4. Deployment:
 - Dockerfiles for api and worker, multi-stage, non-root user
 - The ONNX models baked into the worker image, not downloaded at runtime
 - docker-compose.prod.yml with restart policies and resource limits
 - Caddy or nginx reverse proxy with TLS
 - A backup script for Postgres, and documented restore steps
 - Structured logs to a file with rotation
 - Basic metrics: jobs queued, running, failed; mean processing time; a /metrics endpoint
5. Operational documentation at docs/RUNBOOK.md: how to deploy, how to roll back, what to do when a job is stuck, how to re-run a failed session, how to rotate secrets, and how to restore from backup.
6. A load sanity check: process 20 sessions concurrently and record memory, CPU and wall time. Document the maximum concurrent sessions the chosen server size supports, and what the failure mode looks like when exceeded.

Phase 10 — Continuous quality assurance

What it does

Everything up to Phase 9 tells you the system worked *once*, on the day you measured it. This phase tells you whether it is still working in November, and catches the ways it silently degrades.

Face recognition systems do not fail loudly. They drift. Students change appearance, a teacher starts shooting from a different position, someone changes a threshold, a library update alters preprocessing. Accuracy slides from 94 to 81 percent over a semester and nobody notices until a student disputes their attendance shortage in front of the HOD.

Four defences, in order of value:

Golden dataset regression. Freeze three fully-labelled sessions. Any change to code or parameters re-runs them and compares metrics against a stored baseline. If recall drops more than one percentage point, the change is rejected. This turns “I think that refactor was safe” into a fact, and it is the single most valuable engineering practice in the entire project.

Drift monitoring. Track the mean confident-match similarity per session over time. A steady decline means enrollment photos are going stale. Track override rate per teacher — a rising rate means their capture technique has changed.

Anomaly detection. A teacher recording 100 percent attendance daily while the department averages 86 percent is worth a quiet flag. So is a session where the override count is unusually high, or a student whose similarity scores are consistently near the threshold.

Random audit. Once a week, one committed session is sampled and a human recounts it against the video. This is the only check that catches errors your metrics are blind to, because it does not depend on your assumptions being right.

Integration contract

- Consumes: committed sessions, the golden dataset, historical metrics.
- Produces: a CI gate, a quality dashboard, and alerts.

Definition of done

CI fails on a deliberately introduced regression. The dashboard shows four weeks of trend data. One audit has been completed and its findings written down.

Prompt

PHASE 10 – CONTINUOUS QUALITY ASSURANCE AND MONITORING

DELIVERABLES

1. GOLDEN DATASET REGRESSION SUITE

eval/golden/ contains three fully labelled sessions chosen to differ: one good-quality capture, one marginal, one difficult (poor light or a crowded back row). Store the videos outside git (they are biometric data) and reference them by a documented local path or private storage URI. Commit only the truth.csv files and the baseline metrics.

eval/scripts/regression.py runs the full pipeline over all three with the current params and compares against eval/golden/baseline.json, which stores per-session and aggregate precision, recall, F1, false positives, false negatives, cluster count and processing time.

FAIL the run if any of:

- aggregate recall drops more than 0.01 below baseline
- aggregate precision drops more than 0.01 below baseline
- false positives increase at all
- processing time increases more than 25 percent

Print a clear before/after table on failure. Provide a documented command to deliberately re-baseline after an intentional improvement, which must require an explicit flag and must write who re-baselined it and why into the file.

Wire this into CI so it runs on every pull request. This is the deliverable that makes the rest of the project maintainable.

2. DRIFT MONITORING

A weekly job computing and storing, per course and overall:

- mean and p10 confident-match similarity
- coverage_percent (students found / students enrolled)
- teacher override rate
- proportion of sessions with a preflight warn or fail
- mean processing time

Store in a quality_metric table with a week bucket. Alert when:

- mean confident similarity drops more than 0.04 below its trailing 8-week mean, which usually means enrollment photos are stale
- coverage_percent drops more than 5 points, which usually means capture technique changed
- a single teacher's override rate exceeds twice the department median

For the stale-enrollment case, produce an actionable list of the specific students whose similarity has declined most, so they can be re-enrolled rather than re-enrolling everyone.

3. ENROLLMENT HEALTH CHECK

A monthly job re-running the gallery_sanity analysis from Phase 1 and flagging for re-enrollment any student who now has:

- within-student mean similarity below 0.55
- fewer than gallery_min_embeddings valid embeddings
- enrollment older than 12 months
- a nearest-neighbour similarity to a DIFFERENT student above 0.45, which indicates a genuine look-alike pair that will cause confusions and needs both students re-enrolled with more photos

Surface these on an admin screen as a re-enrollment worklist.

4. ANOMALY DETECTION

Per committed session, compute an anomaly score and flag for review when:

- attendance percent deviates more than 2.5 standard deviations from that course's trailing mean
- a teacher records 100 percent attendance for more than 5 consecutive sessions while the department mean is below 92 percent
- teacher_override_count exceeds 25 percent of the roster
- the same student is in the "not found" band for more than 4 consecutive sessions while being overridden to present each time, which means their enrollment is broken, not that they are absent

Flags go to a department admin screen, phrased as observations rather than accusations. Never auto-reject a session or notify anyone outside the department based on a flag.

5. RANDOM AUDIT WORKFLOW

A weekly job selecting one committed session at random, weighted toward sessions with lower session health. It creates an audit task showing the video alongside the committed roster, with a simple interface to mark each

student agree or disagree. Store results in an `audit_result` table and report audit-measured accuracy separately from pipeline-measured accuracy.

These two numbers diverging is the most important signal the system can produce, because it means your evaluation assumptions are wrong.

6. DATA INTEGRITY CHECKS

A nightly job asserting and alerting on violations of:

- every committed session has exactly `total_enrolled` attendance rows
- no `attendance_record` has been updated or deleted (compare row count and max id against a running log)
- every student with gallery embeddings has valid unrevoked consent
- no `gallery_embedding` exists past its `retention_expires_at`
- `session_summary` counts match a recomputation from `current_attendance`
- no orphaned job artifacts on disk for jobs older than 30 days

7. QUALITY DASHBOARD

A single admin page showing: the four-week trend lines for the drift metrics, the current golden-dataset baseline versus latest run, open anomaly flags, the re-enrollment worklist, recent audit results, and any failing integrity check.

One page. If a maintainer cannot tell in ten seconds whether the system is healthy, the page has failed.

7. What good looks like

Targets to hold yourself to. These are honest, not aspirational.

Metric	Weak	Acceptable	Strong
Student-level recall (present correctly found)	under 0.85	0.90 - 0.95	above 0.96
False positives per session (absent marked present)	3 or more	1 - 2	under 1
Present students with zero accepted crops	over 8 percent	3 - 8 percent	under 3 percent
Back-row (row 5+) recall	under 0.70	0.80 - 0.90	above 0.92
Teacher review time, 90 students	over 2 minutes	45 - 90 seconds	under 45 seconds
End-to-end processing, 60s 4K video, CPU	over 30 min	10 - 25 min	under 10 min
Unusable videos caught by pre-flight, not after processing	under 50 percent	70 - 90 percent	above 90 percent
Gap between audit-measured and pipeline-measured accuracy	over 6 points	2 - 6 points	under 2 points

If back-row recall is your weak number — and it usually is — the fix is almost always capture, not code. Higher camera position, slower pan, 4K, second pass. Re-shoot before you re-tune.

8. Ethics checklist

Complete every item before recording your first classroom video. This is not bureaucracy; it is the difference between a project you can present proudly and one that gets shut down.

- ☐ Written informed consent from every student whose face is enrolled, in a language they read fluently
- ☐ Consent is specific: it names this project, this purpose, this retention period
- ☐ A working, easy revocation route, and a tested deletion path that actually removes the data
- ☐ Your project supervisor and the department head know what you are doing and have approved it
- ☐ Any student under 18 has verifiable parental consent, or is excluded from the study
- ☐ Videos and embeddings are stored encrypted, on a machine you control, never in a public bucket, never in a shared Colab notebook, never committed to git
- ☐ A stated retention period, with automated deletion that you have verified works
- ☐ Nobody is disadvantaged by the system during development — during the entire study, the official attendance remains whatever the teacher records by the normal method. Your system runs in parallel and its output affects nobody's grade

- ☐ Your evaluation report states accuracy differences across skin tone, gender and eyewear where your sample allows, and acknowledges the limitation where it does not

That last point deserves emphasis. Face recognition accuracy is known to vary across demographic groups. If your system is deployed and it is quieter about some students than others, those students bear the cost. Measuring it is the minimum responsible action, and reporting it honestly is what separates engineering from demo-building.

9. Where this goes next

If the evaluation numbers come out well, the interesting result is not the product — it is the ablation table from Phase 7. Almost nobody publishes honest, stratified accuracy figures for group-photo classroom recognition under real conditions, and the commercial market has largely avoided the problem by retreating to terminal-based one-at-a-time scanning.

A clean write-up of “what actually works and what does not, measured properly, on 90 real students” is a stronger portfolio artifact than a working app, and a stronger foundation for a product than any number of features.

Build it, measure it honestly, and let the number decide what happens next.